# Proposal: Building AmberDB – a Minimal Distributed SQL Database from Scratch

**Course:** Distributed Systems (CSEN 317, Spring 2025)
**Team:** Muhammad Haider Shahid & Dishank Kailash Oza

---

## 1 Problem Statement

Companies make use of databases for their applications. Applications today have several requirements of databases. With the increase in users, data, and traffic, it is expected for the database to scale linearly. Another important requirement is for the database to remain available in the event a system failure occurs. There are solutions present today that meet these demands but the inner workings of it are hidden. In this project, we implement a minimal-feature SQL database that meets the requirements of scaling horizontally, dealing with multi-version concurrency control and fault tolerance.

---

## 2 Motivation

We are interested in building each layer and distributed subsystem to gain a hands-on approach to building systems from scratch and not relying on external libraries which abstracts much of the inner workings away. Recent papers indicate that there has been a surge in popularity in cloud native geo-replicated databases. Wanting to design a subset of this type of system would allow us to be exposed to technologies that are trending today. This would benefit and serve as a baseline for students and tech professionals who want to implement their own version of this system and learn techniques such as automatic resharding, MVCC, and Raft based leader election.

---

## 3 Related Work (Literature Review)

**CockroachDB: The Resilient Geo-Distributed SQL Database** (SIGMOD 2020) ACM Digital Library: It utilizes range based sharding with data being divided into key ranges. We learn how to make use of contiguous key ranges as replication units and make use of HLCs to facilitate assigning MVCC timestamps.

**Spanner: Becoming a SQL System** (SIGMOD 2017) ACM Digital Library: We adapt the two-phase commit and read safety timestamps to simulate strong consistency.

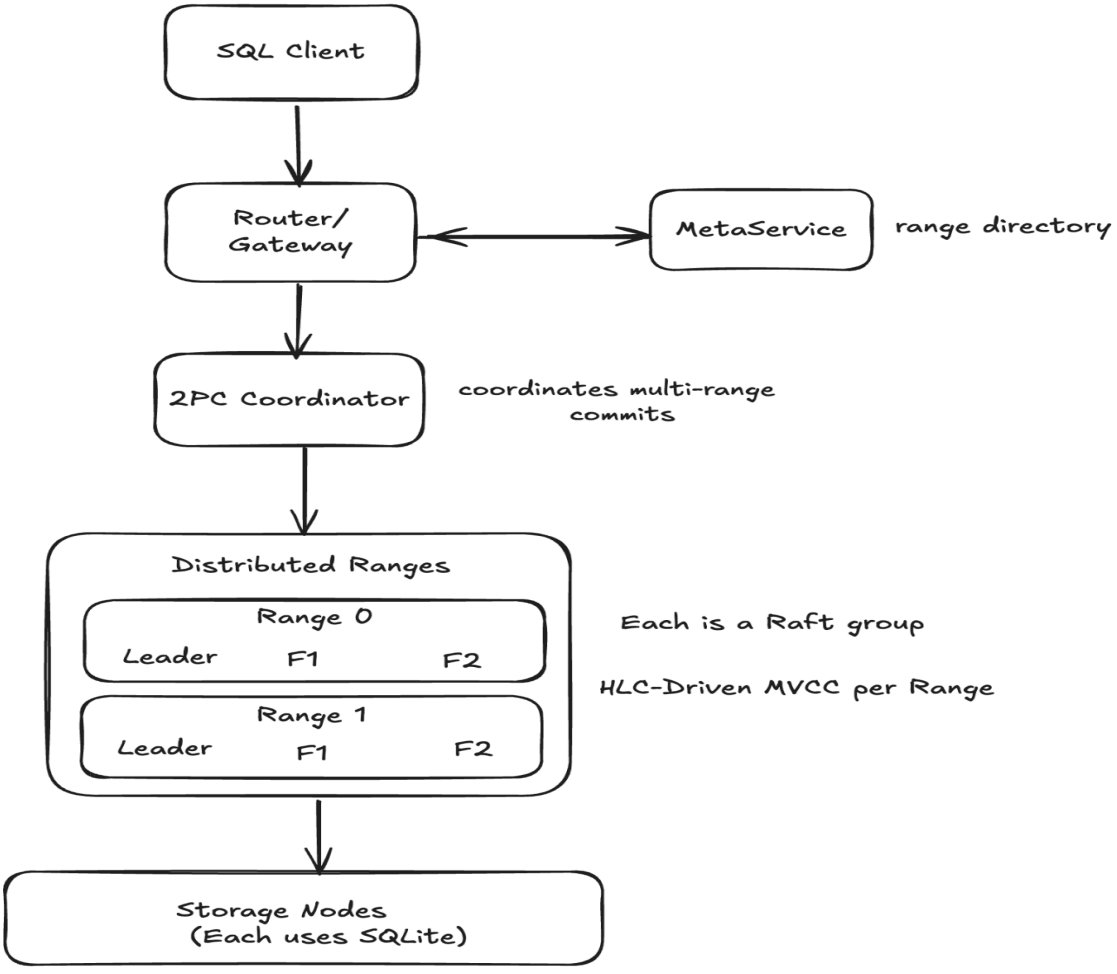
**FoundationDB: A Distributed Unbundled Transactional Key-Value Store** (CACM 2023, original SIGMOD 2021) ACM Digital Library: The transaction processing layer is separated from the storage layer. When it comes to scalability, it is helpful to maintain a thin storage engine.

**PolarDB-SCC: A Cloud-Native Database Ensuring Low-Latency Strongly Consistent Reads** (VLDB 2023) ACM Digital Library: Consistent reads can be provided with minimal coordination and can help in latency and scaling.

---

## 4 Methodology & System Design

### 4.1 High-Level Architecture




Legend:
- F1, F2 = Follower replicas
- MetaService = Tracks range-to-node mapping, QPS, and safe timestamps
- 2PC = Two-Phase Commit coordinator
- HLC = Hybrid Logical Clock used for timestamping writes/reads

Our methodology involves sharding and resharding mechanisms where fixed-size key ranges are split when a QPS (queries per second) or size threshold is exceeded. To facilitate rebalancing and locate key ranges, we use a centralized MetaService, which acts as a metadata directory maintaining mappings from key ranges to their leaders and replicas, as well as tracking load and safe read timestamps. Each query is routed through a gateway that consults the MetaService to direct requests to the appropriate range leader. We use Hybrid Logical Clocks (HLC) to assign timestamps to all writes and reads; each range leader issues timestamps, and replicas maintain a safe timestamp below which follower reads can be safely served. Raft ensures consensus and replication across each key-range (Raft group), and a Two-Phase Commit (2PC) protocol coordinates atomic cross-range transactions. To prevent long-term blocking due to abandoned or stalled transactions, we adopt a Txn-Pusher mechanism inspired by CockroachDB, which attempts to “push” conflicting transactions forward—either by waiting, forcing a timestamp bump, or aborting the stale transaction if it is unresponsive. Raft’s log-based design also ensures that crashed nodes can recover their state and rejoin the system by using the log. In addition, each key in the database maintains multiple versions identified by their HLC timestamps, enabling concurrent reads and writes through MVCC. This ensures that readers can operate on consistent snapshots without interfering with in-flight transactions. The system also supports rebalancing, where range leadership may be transferred and replicas are redistributed. These mechanisms collectively provide fault tolerance, data consistency, and horizontal scalability.

### 4.2 Algorithms & Components

| Component | Algorithm / Technique |
|---|---|
| Leader election & replication | Raft consensus algorithm: Ensures a single leader per range, supports log replication, and guarantees that committed entries are durable and applied in order. Followers can catch up by replaying logs. |
| Version control (MVCC) | Multi-Version Concurrency Control (MVCC): Each key maintains multiple versions with associated HLC timestamps. Readers access the latest committed version ≤ read timestamp. Writers append a new version at commit. |
| Clock synchronization | Hybrid Logical Clocks (HLC): Combines physical (NTP) time and a logical counter. Ensures causality through HLC updates during inter-node communication. Maintains monotonic time even under clock skew. |

| | |
|---|---|
| 2PC Coordinator | Two-Phase Commit (2PC): A consensus algorithm where it makes sure that, across all nodes, a transaction either commits or aborts. |
| MetaService | Acts like a directory: tells the router which range a key belongs to, and which node is the leader |
| Load balancing | Greedy reallocation based on size and QPS thresholds |
| Query processing | ANTLR parser → logical plan → key-value operations |
| Range Resharding | Splits key ranges exceeding QPS/size thresholds to ensure balanced load distribution |
| Txn-Pusher | Detects abandoned or conflicting transactions and pushes or aborts them to ensure liveness |
| Node Recovery | Uses Raft log replay to resync crashed or restarted nodes |

---

## 5 Milestones

| Date (2025) | Milestone |
|---|---|
| **May 05** | Local SQLite wrapper with MVCC tables |
| **May 12** | Raft library integrated |
| **May 19** | MetaStore & dynamic sharding |
| **May 26** | Distributed transaction coordinator & 2PC |
| **Jun 02** | Follower-read support & SQL gateway |

| | |
|---|---|
| **Jun 05** | **Presentation** |
| **Jun 06** | **Final report** |

---

## 6 Resources

- **Hardware / Cloud:** Virtual machine or cloud instances for distributed environment simulation.
- **Software:** Go programming language for implementation (maybe Rust involved). SQLite for storage engine and Git for version control.

---

## 7 References


1. Rebecca Taft et al. *CockroachDB: The Resilient Geo-Distributed SQL Database*. SIGMOD 2020 ACM Digital Library
2. James C. Corbett et al. *Spanner: Becoming a SQL System*. SIGMOD 2017 ACM Digital Library
3. Jingyu Zhou et al. *FoundationDB: A Distributed Key-Value Store*. CACM 2023 ACM Digital Library
4. Yang Jiang et al. *PolarDB-SCC: A Cloud-Native Database Ensuring Low-Latency Strongly Consistent Reads*. PVLDB 16(12) 2023 ACM Digital Library


---